CAPTCHALAB AI

Architecture, Assessment Coverage & Cost Summary

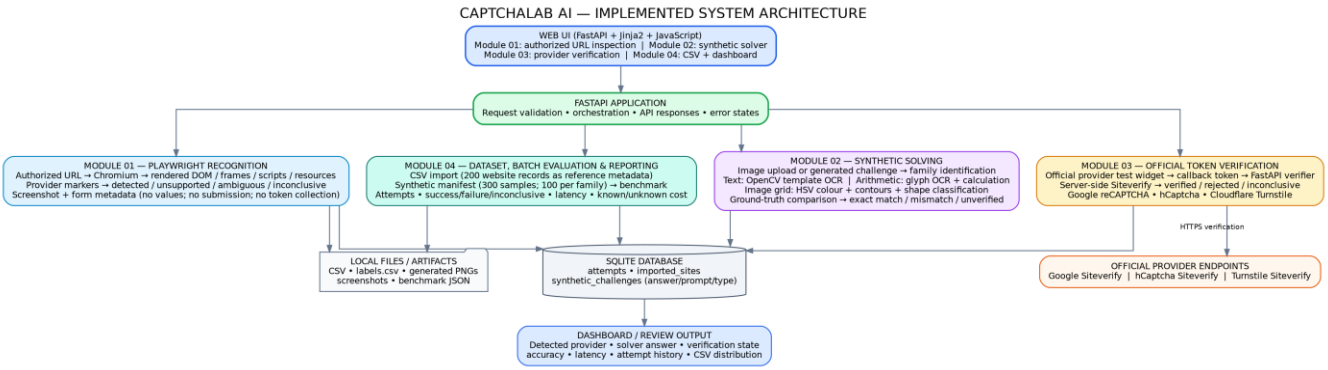
Purpose. CaptchaLab AI is a local-first CAPTCHA recognition, synthetic challenge-solving and official provider-verification application built for the BrainX Technology technical assessment. It combines a FastAPI web application, Playwright-based inspection of authorized pages, CPU-based computer vision, provider verification APIs, SQLite storage and a reporting dashboard.

System architecture at a glance

The browser interface sends requests to FastAPI. The backend validates inputs and routes each task to the appropriate module. Results and attempt metadata are stored in SQLite and presented in the dashboard.

Layer	Responsibility
Web interface	Accepts an authorized URL, challenge image, provider selection or CSV file; displays results and metrics.
FastAPI backend	Validates requests, coordinates modules, returns structured results and handles errors.
Processing services	Playwright inspects pages; synthetic solvers process controlled images; provider adapters call official verification endpoints.
Data and reporting	SQLite stores attempts, imported-site metadata and synthetic challenge ground truth; dashboard reports outcomes, latency and distributions.

SYSTEM ARCHITECTURE DIAGRAM



Operating principle. The application reports only what it can verify. Unknown or incomplete inspections are marked unsupported or inconclusive rather than being presented as successful CAPTCHA-free results.

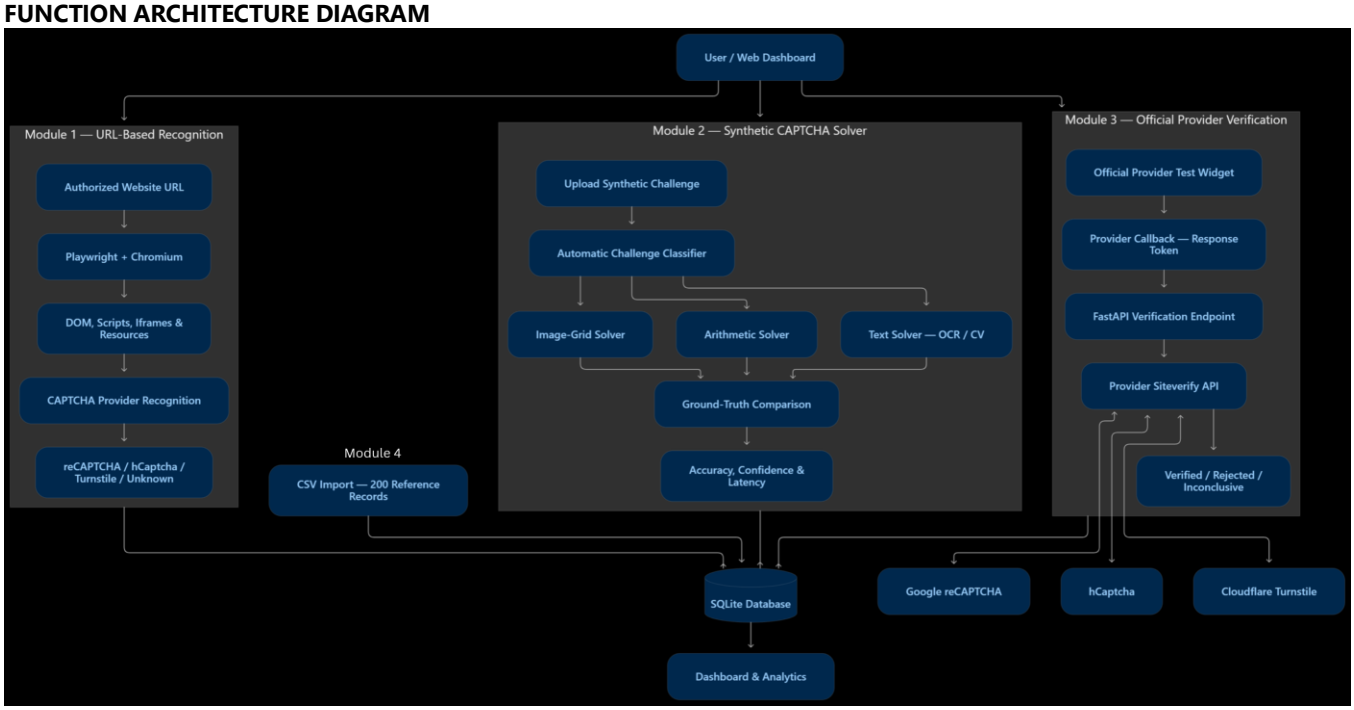
How the modules work

A concise explanation of each workflow and why it exists.

Module	What it does	Why / boundary
Module 01: URL recognition	Input: an authorized website URL. Playwright opens it in Chromium, waits for rendering and inspects accessible DOM elements, frames, scripts and resource URLs. Provider rules identify reCAPTCHA, hCaptcha, Turnstile, unknown, ambiguous or inconclusive states.	Why: provider widgets are often loaded dynamically, so inspecting rendered browser content is more useful than relying only on static HTML. It does not submit contact forms, harvest tokens or crawl unrelated sites.
Module 02: Synthetic solving	Input: an uploaded or generated synthetic image. The classifier routes it to one of three controlled solvers: text (OCR/computer vision), arithmetic (expression reading and calculation), or image-grid (colour/shape detection). Known generated samples are compared with ground truth.	Why: this demonstrates custom solving with measurable accuracy without a GPU or paid solving service. The supported synthetic families are bounded; this is not a claim to solve every production CAPTCHA.
Module 03: Official token verification	A provider test widget issues a response token through its callback. The frontend sends that token to FastAPI. The backend sends it to the matching provider verification endpoint using a server-side secret and reports verified, rejected or inconclusive.	Why: a browser callback alone is not proof of validity. The provider must validate the token server-side. Secrets and raw tokens must not appear in logs. A timeout is inconclusive, not a provider rejection.
Module 04: CSV, batch evaluation & reporting	Imports the interviewer CSV as reference metadata, maintains the synthetic manifest and benchmark results, and displays attempt totals, outcomes, challenge types, methods, processing time, accuracy and cost where known.	Why: it gives the evaluator reproducible data and a clear audit trail. Importing the 200 records does not itself authorize scanning all listed websites.

Functional flow. Recognition, synthetic solving and provider verification are separate workflows. They share the backend, database and dashboard, but one does not replace another.

FUNCTION ARCHITECTURE DIAGRAM



Why access is restricted. The URL allowlist limits server-side browser navigation and reduces SSRF, private-network access and unintended traffic risks. External targets should be added only when owned or explicitly authorized. The assignment excludes bulk contact-form submissions and testing unrelated third-party sites.

Assessment coverage & cost breakdown

Mapped to Assignment 01: CAPTCHA Solving & Verification Tool.

Core evaluation criteria

Assessment criterion	How the implementation addresses it
Recognition / solving accuracy (30%)	Module 01 identifies the three named providers in controlled integrations. Module 02 identifies and solves text, arithmetic and image-grid synthetic challenges; unknown cases are not marked as solved.
Verification-token handling (20%)	Module 03 obtains test-widget tokens and verifies them server-side with Google, hCaptcha and Turnstile. Missing, invalid and provider-reported expired/reused cases return structured outcomes.
Custom AI / authorized API integration (20%)	OpenCV-based local solvers provide custom processing; provider adapters call official HTTPS verification APIs. No paid CAPTCHA-solving API is required. Gemini is optional.
Reliability & error handling (15%)	Input validation, URL allowlisting, timeouts and structured unsupported/inconclusive/error states. Transport failures are distinct from provider rejections.
Interface & reporting (10%)	Web UI for upload/selection and processing; reports attempts, outcomes, challenge type, method/provider, latency and cost where known.
Code quality & documentation (5%)	Modular Python/FastAPI services, environment configuration, setup instructions, tests and this architecture/cost summary support reproduction.

Bonus features

Bonus item	Coverage / qualification
Multiple types/providers	Implemented: three synthetic families plus three named provider integrations.
Fallback handling	Partial: unsupported/low-confidence cases fail safely; no universal alternate-solver fallback is claimed.
Batch & accuracy statistics	Benchmark supports the 300-sample manifest. The generator creates labelled PNGs on demand; generate/include them for an offline demo.
Reusable API endpoint	FastAPI routes support controlled/authorized integrations; public deployment would need authentication, rate limits and hardening.

Cost breakdown

Service	Cost / note
Local stack: Python, FastAPI, Playwright/Chromium, OpenCV, SQLite	Open-source/local components: $0 software licence cost. Local compute, electricity and internet are excluded.
Official provider test workflows	Test credentials are for development/testing; no paid CAPTCHA-solving API is used. Production terms may differ.
Gemini API (optional)	Free within the applicable free-tier quota; beyond it, charges depend on model and token use. Check Google pricing before enabling paid billing.